Question Bank

Class : TE Sub : Internet Programming

Module 5

1. PHP-MySQL Connectivity

There are 3 types of methods in PHP to connect MySQL database through the backend:

1. MySQL
2. MySQLi
3. PDO

mysql() is now obsolete because of security issues like SQL injection etc, but the other two are being actively used.

1. MySQLi

MySQLi is an API that serves as a connector function, linking the backend of PHP applications to MySQL databases. It is an improvement over its predecessor, offering enhanced safety, speed, and a more extensive set of functions and extensions.MySQLi was introduced with PHP 5.0.0, and its drivers were installed in version 5.3.0. The API was designed to support MySQL versions 4.1.13 and newer.

2. PDO

The PHP Data Objects (PDO) extension is a Database Abstraction Layer that serves as an interface for the backend to interact with MySQL databases. It allows for changes to be made to the database without altering the PHP code and provides the flexibility to work with multiple databases. One of the significant advantages of using PDO is that it keeps your code simple and portable.

MySQLi is a PHP extension specifically designed to work with MySQL databases. It offers both procedural and object-oriented interfaces, making it easy to transition from the older MySQL extension. MySQLi also provides support for prepared statements and transactions, which can help improve security and performance. Here's a simple example how to use MySQLi:

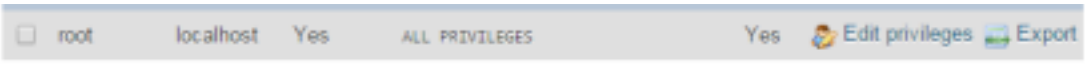
```
<?php
// MySQLi connection
$mysqli = new mysqli("localhost", "username", "password", "database");

// Check connection
if ($mysqli->connect_error) {
die("Connection failed: " . $mysqli->connect_error);
}

// SQL to create table
$sql = "CREATE TABLE users (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
)";
```

```
18.
19. // Execute query
20. if ($mysqli->query($sql) === TRUE) {
21. echo "Table 'users' created successfully";
22. } else {
23. echo "Error creating table: " . $mysqli->error;
24. }
25.
26. // Close connection
27. $mysqli->close();
28. ?>
```

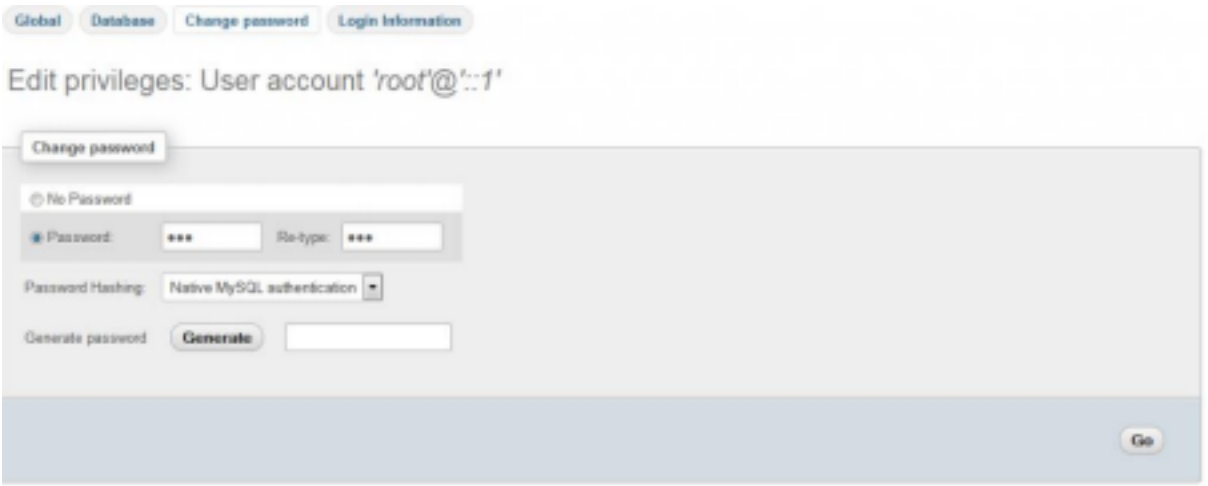
Create MySQL Database at Localhost

Before you start building a PHP connection to a MySQL database, you need to know what PHPMyAdmin is. It's a control panel from which you can manage the database you've created. Open your browser, go to **localhost/PHPMyAdmin**, or click **Admin** in XAMPP UI.

After installing XAMPP, you must add a password to your account for added security. To do this, navigate to the User Account section and locate the username shown in the image below. From there, you can set a password for your account.



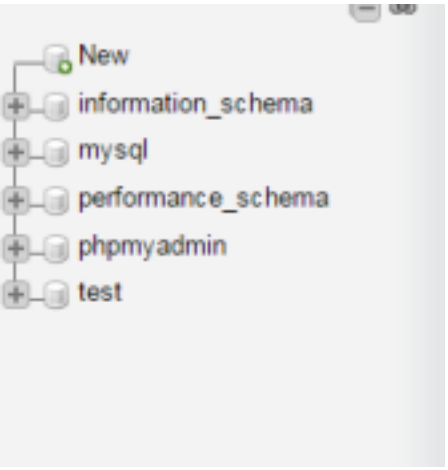
To set a password for your XAMPP account, click on the Edit Privileges button and navigate to the Change Admin Password section. Enter your password and click Save to update your account. Be sure to remember this password, as you'll need it to connect to your database.



Note: While it is not strictly necessary to change the password for accessing databases on your localhost, it is considered good practice to do so. For this reason, we have chosen to use a password for added security.

1. Create Database

Now return to the homepage of PHPMyAdmin. Click the New button to create a new database.



In the new window, enter a name for your database. For this tutorial, we'll name it **"practice"**. Next, select **utf8_general_ci** as the collation, as this will handle all the queries and data covered in this tutorial series. Finally, click on **Create** to create your new database.

Currently, the newly created database is empty because it contains no tables. In the upcoming series, we'll learn how to create tables and insert data into them. Let's focus on connecting the database to localhost using PHP.

2. Create a Folder in htdocs

First, navigate to the XAMPP installation folder and open the **htdocs** subfolder (usually located at **C:\xampp**). Inside **htdocs**, create a new folder named **"practice"** where we'll store our web files. We must create a folder in **htdocs** because XAMPP uses the folders within **htdocs** to execute and run PHP sites.

Note: If you're using WAMP instead of XAMPP, make sure to create the practice folder within the **C:\wamp\www** directory.

3. Create Database Connection File in PHP

Create a new file named **db_connection.php** and save it as a PHP file. We need a separate file for the database connection because it allows us to reuse the same connection code across multiple files. If several files need to interact with the database, you can simply include the **db_connection.php** file instead of writing the connection code multiple times.

```
1. <?php
2. function OpenCon()
3. {
4. $dbhost = "localhost";
5. $dbuser = "root";
6. $dbpass = "1234";
7. $db = "example";
8. $conn = new mysqli($dbhost, $dbuser, $dbpass,$dbname) or die("Connect failed:
     %s\n". $conn -> error);
9. return $conn;
10. }
```

```
11. function CloseCon($conn)
12. {
13. $conn -> close();
14. }
15. ?>
```

Let's take a closer look at the variables used in our **db_connection.php** file and their purpose:

1. **$dbhost:** This variable specifies the host where your RAID server is running. It's usually set to localhost.
2. **$dbuser:** This variable specifies the username for accessing the database. For example, it could be set to root.
3. **$dbpass:** This variable specifies the database password. It should be the same password you use to access PHPMyAdmin.
4. **$dbname:** This variable specifies the database name you want to connect to. In this tutorial, we created a database with a specific name, so you should use that name here.

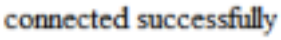
To use the database connection in your code, simply include the **connection.php** file at the top of your script using PHP's include function (e.g., **include 'connection.php'**). This allows you to call and use the connection functions throughout your code. If you need to change the connection details, you only have to update them in one place, and the changes will automatically apply to all other files that include the **connection.php** file.

4. Check Database Connection

To connect to your database, create a new PHP file named **index.php** and add the following code to it:

```
1. <?php
2. include 'db_connection.php';
3. $conn = OpenCon();
4. echo "Connected Successfully";
5. CloseCon($conn);
6. ?>
```

To view the index page, open your web browser and navigate to **localhost/practice/index.php**. You should see the following screen:



2. Explain XML DTD. State its types and describe them.

The XML Document Type Declaration, commonly known as DTD, is a way to describe XML language precisely. DTDs check vocabulary and validity of the structure of XML documents against grammatical rules of appropriate XML language. An XML DTD can be either specified inside the document, or it can be kept in a separate document and then liked separately.

Syntax

Basic syntax of a DTD is as follows −

```
<!DOCTYPE element DTD identifier
[
 declaration1
 declaration2
 ........
]>
```

In the above syntax,

• The **DTD** starts with <!DOCTYPE delimiter.
• An **element** tells the parser to parse the document from the specified root element. • **DTD identifier** is an identifier for the document type definition, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called **External Subset.**
• **The square brackets [ ]** enclose an optional list of entity declarations called *Internal Subset*.

Internal DTD

A DTD is referred to as an internal DTD if elements are declared within the XML files. To refer it as internal DTD, *standalone* attribute in XML declaration must be set to **yes**. This means, the declaration works independent of an external source. Syntax Following is the syntax of internal DTD −

<!DOCTYPE root-element [element-declarations]>

where *root-element* is the name of root element and *element-declarations* is where you declare the elements.

Example

Following is a simple example of internal DTD −

```
<?xml version = "1.0" encoding = "UTF-8" standalone = "yes" ?>
<!DOCTYPE address [
 <!ELEMENT address (name,company,phone)>
 <!ELEMENT name (#PCDATA)>
 <!ELEMENT company (#PCDATA)>
 <!ELEMENT phone (#PCDATA)>
]>

<address>
 <name>Tanmay Patil</name>
 <company>TutorialsPoint</company>
 <phone>(011) 123-4567</phone>
</address>
```

Let us go through the above code −

Start Declaration − Begin the XML declaration with the following statement.

<?xml version = "1.0" encoding = "UTF-8" standalone = "yes" ?>

DTD − Immediately after the XML header, the *document type declaration* follows, commonly referred to as the DOCTYPE −

<!DOCTYPE address [

The DOCTYPE declaration has an exclamation mark (!) at the start of the element name. The DOCTYPE informs the parser that a DTD is associated with this XML document.

DTD Body − The DOCTYPE declaration is followed by body of the DTD, where you declare elements, attributes, entities, and notations.

```
<!ELEMENT address (name,company,phone)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT company (#PCDATA)>
<!ELEMENT phone_no (#PCDATA)>
```

Several elements are declared here that make up the vocabulary of the <name> document. <!ELEMENT name (#PCDATA)> defines the element *name* to be of type "#PCDATA". Here #PCDATA means parse-able text data.

End Declaration − Finally, the declaration section of the DTD is closed using a closing bracket and a closing angle bracket (**]>**). This effectively ends the definition,

and thereafter, the XML document follows immediately.

Rules

• The document type declaration must appear at the start of the document (preceded only by the XML header) − it is not permitted anywhere else within the document. • Similar to the DOCTYPE declaration, the element declarations must start with an exclamation mark.

• The Name in the document type declaration must match the element type of the root element.

XML document with an internal DTD

```
<?xml version="1.0"?>
<!DOCTYPE address [
<!ELEMENT address (name, email, phone, birthday)>
<!ELEMENT name (first, last)>
<!ELEMENT first (#PCDATA)>
<!ELEMENT last (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
<!ELEMENT birthday (year, month, day)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT month (#PCDATA)>
<!ELEMENT day (#PCDATA)>
]>

<address>
    <name>
        <first>Rohit</first>
        <last>Sharma</last>
    </name>
    <email>sharmarohit@gmail.com</email>
    <phone>9876543210</phone>
    <birthday>
        <year>1987</year>
        <month>June</month>
        <day>23</day>
    </birthday>
</address>
```

External DTD

In external DTD elements are declared outside the XML file. They are accessed by specifying the system attributes which may be either the legal *.dtd* file or a valid URL. To refer it as external DTD, *standalone* attribute in the XML declaration must be set as **no**. This means, declaration includes information from the external source. Syntax Following is the syntax for external DTD −

```
<!DOCTYPE root-element SYSTEM "file-name">
```

where *file-name* is the file with *.dtd* extension.

Example

The following example shows external DTD usage −

```
<?xml version = "1.0" encoding = "UTF-8" standalone = "no" ?>
<!DOCTYPE address SYSTEM "address.dtd">
<address>
 <name>Tanmay Patil</name>
 <company>TutorialsPoint</company>
 <phone>(011) 123-4567</phone>
</address>
```

The content of the DTD file **address.dtd** is as shown −

```
<!ELEMENT address (name,company,phone)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT company (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
```

Example -1 XML document with an external DTD:

```xml
<?xml version="1.0"?>
<!DOCTYPE address SYSTEM "address.dtd">
<address>
    <name>
        <first>Rohit</first>
        <last>Sharma</last>
    </name>
    <email>sharmarohit@gmail.com</email>
    <phone>9876543210</phone>
    <birthday>
        <year>1987</year>
        <month>June</month>
        <day>23</day>
    </birthday>
</address>
```

address.dtd:

- <!ELEMENT address (name, email, phone, birthday)>
- <!ELEMENT name (first, last)>
  - <!ELEMENT first (#PCDATA)>
  - <!ELEMENT last (#PCDATA)>
- <!ELEMENT email (#PCDATA)>
- <!ELEMENT phone (#PCDATA)>
- <!ELEMENT birthday (year, month, day)>
  - <!ELEMENT year (#PCDATA)>
  - <!ELEMENT month (#PCDATA)>
  - <!ELEMENT day (#PCDATA)>

3. Short note: XML Parser

XML parser is a software library or a package that provides interface for client applications to work with XML documents. It checks for proper format of the XML document and may also validate the XML documents. Modern day browsers have built-in XML parsers.

Following diagram shows how XML parser interacts with XML document –

The goal of a parser is to transform XML into a readable code.

To ease the process of parsing, some commercial products are available that facilitate the breakdown of XML document and yield more reliable results.

The most important type is DOM and SAX which is explained detail.

1. DOM Parser (Tree-Based)

Document Object Model is a W3C Standard and converts the XML document which is to parsed into a collection of objects and uses DOM API. It consists of a collection of nodes and is associated with other nodes in the tree. DOM is much easier to use as sorting and searching process is made faster.In DOM parser the content of the XML file is modified with Node and Node List. The Steps involved in Parsing with java:

- Getting document builder objects
- Taking XML document as input, parseit and return the class.
- Getting values of the input id through attributes and sub-elements.
- Display the results.

First is the XML file that generates the values which are going to be parse and java objects are constructed automatically.

Example

new.xml

```
<?xml version="1.0"?>
<shops>
<supermarket>
<sid>201</sid>
<sname>sparc</sname>
<product> grocery</product>
<branch> two</branch>
<location> new york</location>
</supermarket>
<supermarket>
<sid>540</sid>
<sname> big basket</sname>
<product> grocery</product>
<branch> seven</branch>
<location>India</location>
</supermarket>
<supermarket>
<sid>301</sid>
<sname>Wallmart</sname>
<product> grocery</product>
<branch> fifteen</branch>
<location> France</location>
```

```
</supermarket>
</shops>
```

Read.java

```
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;
import org.w3c.dom.Node;
import org.w3c.dom.Element;
import java.io.File;
public class Read
{
public static void main(String argv[])
{
try
{
File file = new File("C:\\Program Files\\Java\\jdk-13.0.1\\par\\new.xml");
DocumentBuilderFactory docb =
DocumentBuilderFactory.newInstance(); DocumentBuilder dbu =
docb.newDocumentBuilder();
Document dt = dbu.parse(file);
dt.getDocumentElement().normalize();
System.out.println("Root: " +
dt.getDocumentElement().getNodeName()); NodeList nd =
dt.getElementsByTagName("supermarket"); for (int i = 0;
i<nd.getLength(); i++)
{
Node node = nd.item(i);
System.out.println("\nNodeName :" +
node.getNodeName()); if (node.getNodeType() ==
Node.ELEMENT_NODE)
{
Element el = (Element) node;
System.out.println("supermarket sid: "+
el.getElementsByTagName("sid").item(0).getTextContent());
System.out.println("sname:
"+ el.getElementsByTagName("sname").item(0).getTextContent());
System.out.println("product:
"+ el.getElementsByTagName("product").item(0).getTextContent());
System.out.println("branch:
"+ el.getElementsByTagName("branch").item(0).getTextContent());
System.out.println("location: "+
el.getElementsByTagName("location").item(0).getTextContent());
}
}
}
catch (Exception e)
{
e.printStackTrace();
}
```

```
}
}
```

2. SAX Parser

SAX Is Simple API for XML and meant has Push Parseralso considered to be stream oriented XML Parser. it is used in case of high- performance applications like where the XML file is too largeand comes with the community- based standard and requires less memory. The main task is to read the XML file and creates an event to do call functionor uses call back routines. The working of this parser is just like Event handler part of the java. it is necessary to register the handlers to parse the document for handing different events. SAX Parser uses three methods startElement() , endElement() , characters().

- **startElement():** Is used to identify the element, start element is identified.
- **endElement():** To stop the Supermarket tag in the example.
- **character():** Is used to identify the character in the node

Next section shows an implementation of parsing using SAX with the java. Here we have XML file new.xml to parse and to create a list of supermarket object.

The xml file is the same file used in DOM Parser new.xml and next step generate Rsax.java file

Example

Rsax.java

```
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;
public class Rsax
{
public static void main(String args[])
{
try
{
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParsersaxParser = factory.newSAXParser();
DefaultHandler handler = new DefaultHandler()
{
booleansid = false;
booleansname = false;
boolean product = false;
boolean branch = false;
boolean location = false;
public void startElement( String sg, String pp,String q, Attributes a) throws
SAXException {
System.out.println("Beginning Node :" + q);
if(q.equalsIgnoreCase("SID"))
{
sid=true;
}
if (q.equalsIgnoreCase("SNAME"))
{
sname = true;
```

```
}
if (q.equalsIgnoreCase("PRODUCT"))
{
product = true;
}
if (q.equalsIgnoreCase("BRANCH"))
{
branch = true;
}
if (q.equalsIgnoreCase("LOCATION"))
{
location = true;
}
}
public void endElement(String u, String l, String qNa) throws
SAXException {
System.out.println("LAst Node:" + qNa);
}
public void characters(char chr[], int st, int len) throws
SAXException {
if (sid)
{
System.out.println("SID : " + new String(chr, st, len));
sid = false;
}
if (sname)
{
System.out.println("Shop Name: " + new String(chr, st, len));
sname = false;
}
if (product)
{
System.out.println("Available Product: " + new String(chr, st, len));
product = false;
}
if (branch)
{
System.out.println("No.of Branches: " + new String(chr, st, len));
branch = false;
}
if (location)
{
System.out.println("Address : " + new String(chr, st, len));
location = false;
}
}
};
saxParser.parse("C:\\Program Files\\Java\\jdk-13.0.1\\par\\new.xml",
handler); }
catch (Exception ex)
{
```

```
ex.printStackTrace();
}
}
}
```

4. How can a cookie be created in PHP script?

PHP Cookies
A cookie is often used to identify a user. A cookie is a small file that the server embeds on the user's computer. Each time the same computer requests a page with a browser, it will send the cookie too. With PHP, you can both create and retrieve cookie values.

Create Cookies With PHP
A cookie is created with the setcookie() function.
Syntax
setcookie(*name, value, expire, path, domain, secure, httponly*);
Only the *name* parameter is required. All other parameters are optional.

PHP Create/Retrieve a Cookie
The following example creates a cookie named "user" with the value "John Doe". The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website (otherwise, select the directory you prefer).
We then retrieve the value of the cookie "user" (using the global variable $_COOKIE). We also use the isset() function to find out if the cookie is set:

ExampleGet your own PHP Server

```
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1
day ?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
 echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
 echo "Cookie '" . $cookie_name . "' is set!<br>";
 echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

Modify a Cookie Value
To modify a cookie, just set (again) the cookie using the setcookie() function:
Example

```
<?php
$cookie_name = "user";
$cookie_value = "Alex Porter";
```

```
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
 echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
 echo "Cookie '" . $cookie_name . "' is set!<br>";
 echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

Delete a Cookie

To delete a cookie, use the setcookie() function with an expiration date in the past:

Example

```
<?php
// set the expiration date to one hour ago
setcookie("user", "", time() - 3600);
?>
<html>
<body>

<?php
echo "Cookie 'user' is deleted.";
?>

</body>
</html>
```

Check if Cookies are Enabled

The following example creates a small script that checks whether cookies are enabled. First, try to create a test cookie with the setcookie() function, then count the $_COOKIE array variable:

Example

```
<?php
setcookie("test_cookie", "test", time() + 3600, '/');
?>
<html>
<body>

<?php
if(count($_COOKIE) > 0) {
 echo "Cookies are enabled.";
} else {
 echo "Cookies are disabled.";
}
?>
```

```
</body>
</html>
```

5. Difference between Variable and constant in php.

PHP Constants: PHP Constants are the identifiers that remain the same. Usually, it does not change during the execution of the script. They are case-sensitive. By default, constant identifiers are always uppercase. Usually, a Constant name starts with an underscore or a letter which is followed by a number of letters and numbers. They are no need to write a constant with the $ sign. The constant() function is used to return the value of a constant.
Example:
PHP

```
<?php
 define("Hello", "Welcome to geeksforgeeks.com!");
 echo Hello;
 ?>
```

Output:
Welcome to geeksforgeeks.com!
PHP Variables: A PHP variable is a name given to a memory address that holds data. The basic method to declare a PHP variable is by using a $ sign which is followed by the variable name. A variable helps the PHP code to store information in the middle of the program. If we use a variable before it is assigned then it already has a default value stored in it. Some of the data types that we can use to construct a variable are integers, doubles, and boolean.

Example:
PHP

```
<?php
 $txt = "Hello from geeksforgeeks!";
 $x = 5;
 $y = 10.5;
 echo $txt;
 echo "<br>";
 echo $x;
 echo "<br>";
 echo $y;
 ?>
```

Output:
Hello from geeksforgeeks!
5
10.5
Difference between PHP Constants and PHP Variables

PHP Constants	PHP Variables
In PHP constants there is no need to use $ sign.	In PHP Variables the $ sign is been used.
The data type of PHP constant cannot be changed during the execution of the script.	The data type of the PHP variable can be changed during the execution of the script.
A PHP constant once defined cannot be redefined.	A PHP variable can be undefined as well as can be redefined.

We can not define a constant using any simple assignment operator rather it can only be defined using define().	We can define a variable using a simple assignment operation(=).
Usually, constants are written in numbers.	On the other hand, variables are written in letters and symbols.
PHP constants are automatically global across the entire script.	PHP variables are not automatically global in the entire script.
PHP constant is comparatively slower than PHP variable	A PHP variable is comparatively faster than the PHP constant

6. State and explain the built-in function in php.

Built-In Functions of PHP

- Built-in functions are the functions that are provided by PHP to make programming more convenient.
- Many built-in functions are present in PHP which can be employed in our code. These functions are very helpful in achieving programming goals.
- The built-in functions in PHP are very simple and easy to use. They make PHP powerful and useful.
- PHP is very rich in terms of Built-in functions. Some of the important Built-in functions of PHP are as follows:

PHP Array Functions

These functions allow interacting with and manipulating arrays in various ways. Arrays are essential for storing, managing, and operating on sets of variables.

- array() - Create an array
- array_chunk() - Splits an array into chunks of arrays
- array_diff() - Compares array values, and returns the differences
- array_fill() - Fills an array with values
- array_intersect() - Compares array values, and returns the matches •

array_merge() - Merges one or more arrays into one array

PHP Calender Functions

The calendar extension presents a series of functions to simplify converting between different calendar formats.

- cal_from_jd() - Returns information about a given calendar.
- cal_from_jd() - Converts a Julian day count into a date of a specified calendar.
- JDMonthName() - Returns a month name.
- JDDayOfWeek() - Returns the day of a week.
- jdtounix() - Converts a Julian day count to a Unix timestamp.
- unixtojd() - Converts a Unix timestamp to a Julian day count.

PHP Character Functions

The functions provided by this extension check whether a character or string falls into a certain character class according to the current locale.

- ctype_alnum() - Check for alphanumeric character(s).
- ctype_alpha() - Check for alphabetic character(s).
- ctype_digit() - Check for numeric character(s).
- ctype_space() - Check for whitespace character(s).
- ctype_upper() - Check for uppercase character(s).
- ctype_lower() - Check for lowercase character(s).

PHP Date & Time Functions

These functions allow getting the date and timing from the server where PHP scripts are running. These functions are used to format the date and time in many different ways.
date_date_set() - Sets the date.

- date_default_timezone_get() – Returns the default time zone.
- date_default_timezone_set() - Sets the default time zone.
- date_format() - Returns date formatted according to given format.
- date_time_set() - Sets the time.
- getdate() - Returns an array that contains date and time information for a Unix timestamp.
- time() - Returns the current time as a Unix timestamp.

PHP Directory Functions

These functions are provided to manipulate any directory. PHP needs to be configured with --enable-chroot-func option to enable chroot() function.

- chdir() - Changes current directory.
- chroot() - Change the root directory.
- dir() - Opens a directory handle and returns an object.
- closedir() - Closes a directory.
- getcwd() - Gets the current working directory.
- opendir() - Open directory handle.

PHP Error Handling Functions

These are functions dealing with error handling and logging. They allow user to define their own error handling rules, as well as modify the way the errors can be logged. This allows user to change and enhance error reporting to suit their needs.

Using these logging functions, user can send messages directly to other machines, to an email, to system logs, etc. So user can selectively log and monitor the most important parts of their applications and websites.

- error_log() - Sends an error to the server error-log, to a file or to a remote destination.
- error_reporting() - Specifies which errors are reported.
- set_error_handler()- Sets a user-defined function to handle errors.
- set_exception_handler() - Sets a user-defined function to handle exceptions.
- error_get_last() - Gets the last error occurred.
- trigger_error() - Creates a user-defined error message.

PHP MySQL Function

PHP MySQLi functions gives to access the MySQLi database servers. PHP works with MySQLi version 4.1.13 or newer.

- mysqli connect - It opens a connection to a MySQLi Server.
- mysqli create db - It is used to create MySQLi connection.
- mysqli close - It is used to close MySQLi connection.
- mysqli ping - It is used to pings a server connection and reconnect to server if connection is lost.
- mysqli query - It performs a query against the database.
- mysqli real connect - This function opens a new connection to the MySQLi. • mysqli refresh - This function refreshes tables or caches, or resets the replication server information.
- mysqli_stat - This function returns the current system status.
- mysqli rollback - This function rolls back the current transaction for the specified database connection.

PHP File System Functions

The file system functions are used to access and manipulate the file system. PHP provides all the possible functions may need to manipulate a file.

- chmod() - Changes file mode.
- file_exists() - Checks whether a file or directory exists.
- file_get_contents() - Reads entire file into a string.
- file_put_contents() - Write a string to a file.
- file() - Reads entire file into an array.
- basename() - Returns filename component of path.

PHP String Functions

PHP string functions are the part of the core. It manipulate strings in almost any possible way. • chop() - It is used to removes whitespace.

- chunk_split() - It is used to split a string into chunks.
- crypt() - It is used to hashing the string.
- str_split() - It is used to convert a string to an array.
- strchr() - It is used to searches for the first occurrence of a string inside another. • trim() - It used to remove the white-spaces and other characters.

PHP Network Functions

PHP Network functions give access to internet from PHP.

- fsockopen() - It is used to open internet or Unix domain socket connections. • gethostbyname() - It is used to get the internet host name which has given by IP Address.
- gethostname() - It is used to get the host name.
- header() - It is used to send the row of HTTP Headers.
- openlog() - It used to open connection to system logger.
- setcookie() - It used to set the cookies.

7. Explain php control structures.

Control Structures in PHP

The Control Structures (or) Statements used to alter the execution process of a

program. These statements are mainly categorized into following:

- Conditional Statements
- Loop Statements

- Jump Statements

Conditional Statements :

Conditional Statements performs different computations or actions depending on conditions. In PHP, the following are conditional statements

- if statement
- if - else statement
- if - elseif - else statement
- switch statement

☞ if statement

The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.

Syntax :

```
if (condtion)
{
        statements
}
```

☞ if - else statement

The if-else statement provides an else block combined with the if statement which is executed in the false case of the condition.

Syntax :

```
if (condtion)
{
        statements
}
else
{
        statements
}
```

☞ if - elseif - else statement

The elseif statement enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.

Syntax :

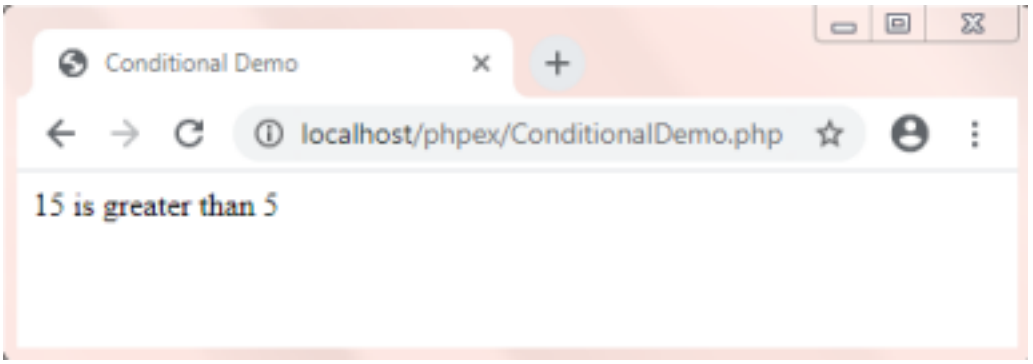
```
if (condtion1)
{
        statements
}
else if (condtion2)
{
        statements
}
else if (condtion3)
{
```

```
        statements
}
.
.
else
{
        statements
}
```

Example: "ConditionalDemo.php"

```
<html>
<head>
<title>Conditional Demo</title>
</head>
<body>
<?php
$x=15;
$y=5;
if ($x > $y)
{
 echo "$x is greater than $y";
}
else if ($x < $y)
{
 echo "$x is lessthan $y";
}
else
{
 echo "Both are Equal";
}
?>
</body>
</html>
```

Output :



☞ switch statement

The switch statement enables us to execute a block of code from multiple conditions depending upon the expression.

Syntax :

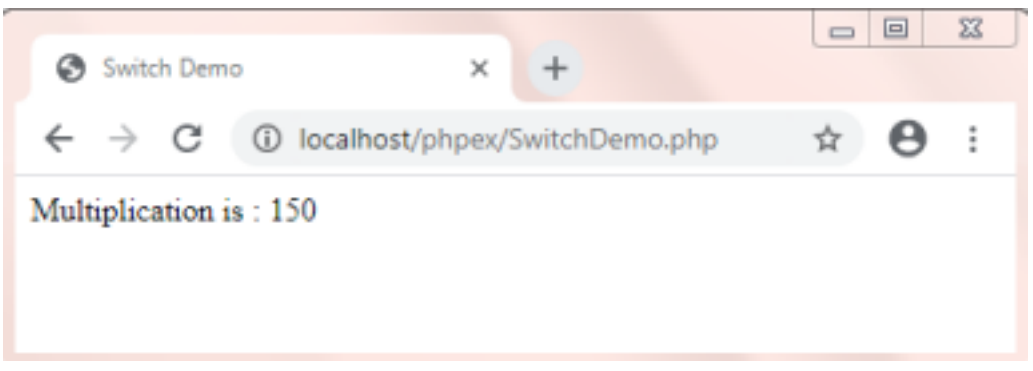
```
switch (expression)
```

```
{
case 1: statements
                break;
case 2: statements
                break;
case 3: statements
                break;
.
.
default: statements
}
```

Example: "SwitchDemo.php"

```
<html>
<head>
<title>Switch Demo</title>
</head>
<body>
<?php
 $x=15;
 $y=10;
 $op='*';
 switch($op)
 {
 case '+': $z = $x + $y;
 echo "Addition is : $z";  break;
 case '-': $z = $x - $y;
 echo "Subtraction is : $z";  break;
 case '*': $z = $x * $y;
 echo "Multiplication is : $z";  break;
 case '/': $z = $x / $y;
 echo "Division is : $z";  break;
 case '%': $z = $x % $y;
 echo "Modulus is : $z";  break;
 default: echo "Invalid Operator";  }
?>
</body>
</html>
```

Output :



Loop Statements :

Sometimes we may need to alter the flow of the program. If the execution of a specific code may need to be repeated several numbers of times then we can go for loop statements.

In PHP, the following are loop statements

- while loop
- do - while loop
- for loop

☞ while loop statement

With the while loop we can execute a set of statements as long as a condition is true. The while loop is mostly used in the case where the number of iterations is not known in advance.

Syntax :

```
while (condition)
{
        statements
}
```

Example: "WhileDemo.php"

```
<html>
<head>
<title>While Demo</title>
</head>
<body>
<h1>While Demo</h1>
<?php
$n=1;
while($n<=5)
{
 echo "$n <br/>";
 $n++;
}
?>
</body>
</html>
```

Output :

☞ do - while loop statement

The do-while loop will always execute a set of statements atleast once and then execute a

set of statements as long as a condition is true.

Syntax :

```
do
{
 statements
} while (condition);
```

Example: "DoWhileDemo.php"

```
<html>
<head>
<title>Do-While Demo</title>
</head>
<body>
<h1>Do-While Demo</h1>
<?php
$n=1;
do
{
 echo "$n <br/>";
 $n++;
} while($n<=5);
?>
</body>
</html>
```

Output :

☞ for loop statement

With the for loop we can execute a set of statements specified number of times. The for loop is mostly used in the case where the number of iterations is known in advance..

Syntax :

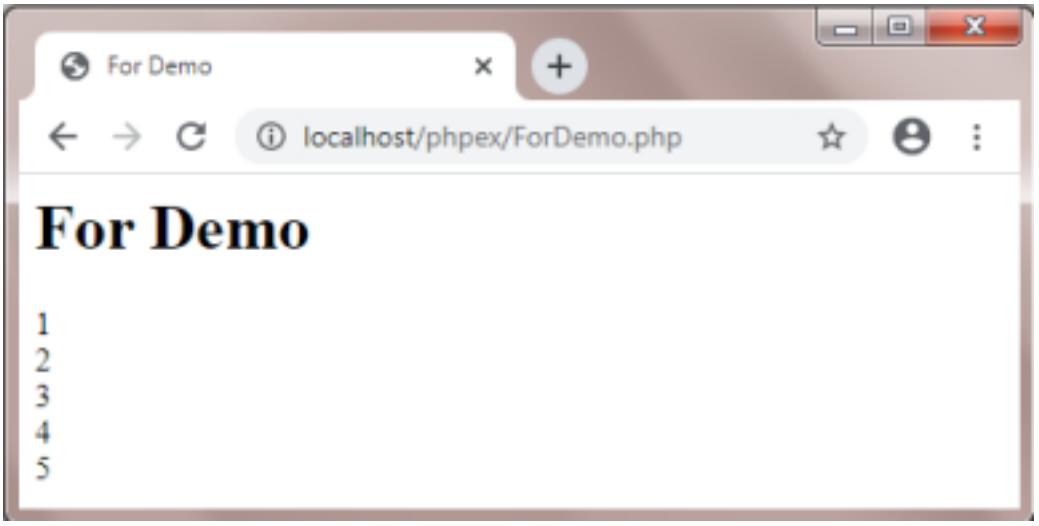
```
for (initialization; condition; increment/decrement)
{
 statements
}
```

Example: "ForDemo.php"

```
<html>
```

```
<head>
<title>For Demo</title>
</head>
<body>
<h1>For Demo</h1>
<?php
for($i=1;$i<=5;$i++)
{
 echo "$i <br/>";
}
?>
</body>
</html>
```

Output :



Jump Statements :

Jump statements in PHP are used to alter the flow of a loop like you want to skip a part of a loop or terminate a loop.

In PHP, the following are jump statements

- break statement
- continue statement

☞ break statement

The break is a keyword in php which is used to bring the program control out of the loop. i.e. when a break statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop.

The break statement breaks the loops one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. The break is commonly used in the cases where we need to break the loop for a given condition.

Syntax :

```
break;
```

Example: "BreakDemo.php"

```
<html>
<head>
<title>Break Demo</title>
</head>
<body>
```

```
<h1>Break Demo</h1>
<?php
for($i=1;$i<=10;$i++)
{
 if($i==5)
 {
 break; //terminates the current loop
 }
 echo "$i <br/>";
}
echo "Loop is Over !";
?>
</body>
</html>
```

Output :

☞ continue statement

The continue statement in php is used to bring the program control to the beginning of the loop. i.e. when a continue statement is encountered inside the loop, remaining statements are skipped and loop proceeds with the next iteration.

The continue statement skips the remaining lines of code inside the loop and start with the next iteration. It is mainly used for a particular condition inside the loop so that we can skip some specific code for a particular condition.

Syntax :

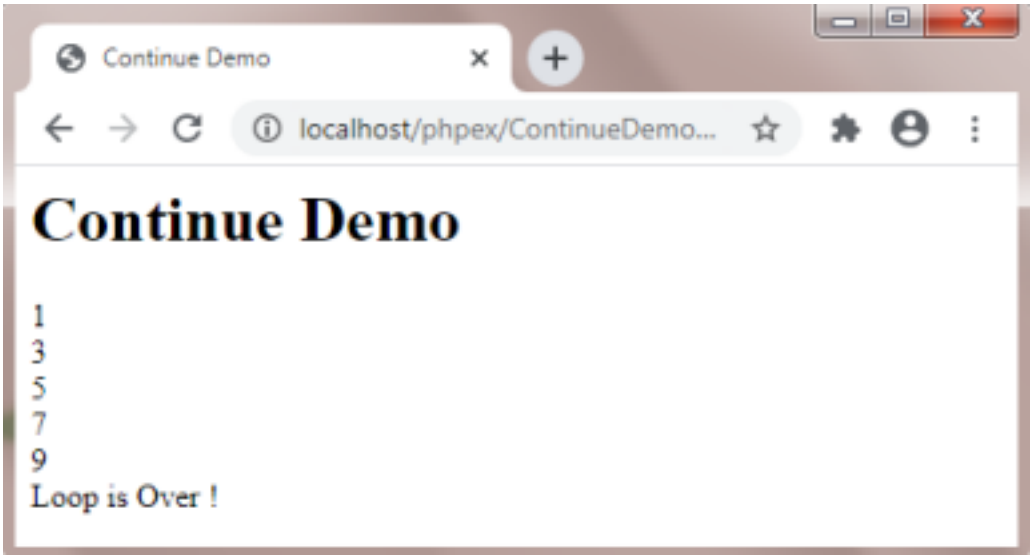
```
continue;
```

Example: "ContinueDemo.php"

```
<html>
<head>
<title>Continue Demo</title>
</head>
<body>
<h1>Continue Demo</h1>
<?php
for($i=1;$i<=10;$i++)
{
 if($i%2==0)
 {
```

```
 continue; //terminates the current Iteration and moves to Next  }
 echo "$i <br/>";
}
echo "Loop is Over !";
?>
</body>
</html>
```

Output :



8. Explain any four String Manipulation functions with examples.

String Manipulation Functions

C++ provides a set of string handling functions to work on strings. These functions are available in string.h header files. The following are few string manipulation functions. They are,

1. strlen() function
2. strcpy() function
3. strcat() function
4. strcmp() function
5. strchr() function
6. strstr() function
7. substr() function

1. strlen() function

strlen() function returns the length of the string. It does not include the null terminator. Syntax

strlen(string1);

Example

strlen(“Hello”);

2. strcpy() Function

strcpy() function is used to copy a string to a string variable.

Syntax

strcpy(string1, string2);

Example

strcpy(name, “helloworld”);

Here, both string1 and string2 are character arrays where, string2 may be a string

variable (or) string constant.

3 strcat() Function

strcat() function appends one string at the end of another string. It returns a single string that consists of characters of string1 followed by string2.

Syntax

strcat(string1, string2);

Example

strcat(“Hello”, “World”);

4. strcmp() Function

strcmp( ) function is used to compare two strings. It returns 0 if string1 and string2 are equal, If string1 is greater than string2 it returns negative number and if string2 is less than string 1, it returns positive number.

Syntax

strcmp(string1, string2);

Example:

strcmp(‘“Hello”, “Help”);

5. strchr() Function

strchr( ) function finds the first occurrence of given character in a string. Syntax

char strchr(const char str, int ch);

Example

strchr(welcome, e);

6. strstr() Function

strstr() function returns a pointer to first occurrence of string2 in string1. Syntax

strstr(string1, string2);

Example

strstr(“estudies”, “foryou”);

7. substr() Function

substr( ) extracts the characters between two specified indices from a string and returns the new sub string.

Syntax

str.substr(start, end);

Example

str = “Hello world”

str.substr(2, 5);

Program

```
#include<iostream>
#include<cstring>
using namespace std;
int main( )
{
int x, lenl, len2;
char str1[] = “siagroup”;
char str2[] = “group”;
cout<<“\nstrlen( )”;
len1= strlen(str1);
```

```
len2= strlen(str2);
cout <<“\nstr1:”<<strl<< “\nlength <<lenl<<“characters”<< endl;
cout <<“\nstr2: ”<<str2<<“\nlength: »<<len2<<“characters”<<endl;
cout<<“\nstrcmp()”;
x= strcmp(str1, str2);
if(x!=0)
{
cout<<“\n\nStrings are not equal \n”;
cout<<“\nstrstr()”;
char *p = strstr(str1,str2);
if (p)
cout<< “\n” <<str2 <<“ is present in ” <<strl <<“ at position” <<p-str1;
else
cout<<“\n” <<str2<<“ is not present ” <<str1<<endl;
cout<<“\nstrcat()”;
strcat(str1, str2);
cout<<“\nstr1 =\t ’<<str1; /*joining strl and str2*/
}
else
cout<<“\n\nStrings are equal \n”;
char* ch = strchr(str1,'i’);
cout<<“\n strchr()”;
cout<<“\n’<<ch - str1+1;
strcpy(str1, str2);
cout<<“\nstrcpy( )”;
cout<<“\nstr1 = \t’<< str1;
return 0;
}
```

9. Explain how form validation works in php with suitable examples.

Form Validation in PHP

An HTML form contains various input fields such as text box, checkbox, radio buttons, submit button, and checklist, etc. These input fields need to be validated, which ensures that the user has entered information in all the required fields and also validates that the information provided by the user is valid and correct.

There is no guarantee that the information provided by the user is always correct. PHP validates the data at the server-side, which is submitted by HTML form. You need to validate a few things:

1. Empty String
2. Validate String
3. Validate Numbers
4. Validate Email
5. Validate URL
6. Input length

Empty String

The code below checks that the field is not empty. If the user leaves the required field empty, it will show an error message. Put these lines of code to validate the required field. 1. **if**

```
(empty empty ($_POST["name"])) {
    2. $errMsg = "Error! You didn't enter the Name.";
    3. echo $errMsg;
    4. } else {
    5. $name = $_POST["name"];
    6. }
```

Validate String

The code below checks that the field will contain only alphabets and whitespace, for example - name. If the name field does not receive valid input from the user, then it will show an error message:

```
1. $name = $_POST ["Name"];
2. if (!preg_match ("/^[a-zA-z]*$/", $name) ) {
3. $ErrMsg = "Only alphabets and whitespace are allowed.";
4. echo $ErrMsg;
5. } else {
6. echo $name;
7. }
```

Validate Number

The below code validates that the field will only contain a numeric value. **For example -** Mobile no. If the *Mobile no* field does not receive numeric data from the user, the code will display an error message:

```
1. $mobileno = $_POST ["Mobile_no"];
2. if (!preg_match ("/^[0-9]*$/", $mobileno) ){
3. $ErrMsg = "Only numeric value is allowed.";
4. echo $ErrMsg;
5. } else {
6. echo $mobileno;
7. }
```

Validate Email

A valid email must contain @ and . symbols. PHP provides various methods to validate the email address. Here, we will use regular expressions to validate the email address. The below code validates the email address provided by the user through HTML form. If the field does not contain a valid email address, then the code will display an error message:

```
1. $email = $_POST ["Email"];
2. $pattern = "^[_a-z0-9-]+(\.[_a-z0-9-]+)*@[a-z0-9-]+(\.[a-z0-9-]+)*(\.[a-z]{2,3})$^";
3. if (!preg_match ($pattern, $email) ){
4. $ErrMsg = "Email is not valid.";
5. echo $ErrMsg;
6. } else {
7. echo "Your valid email address is: " .$email;
8. }
```

Input Length Validation

The input length validation restricts the user to provide the value between the specified

range, for Example - Mobile Number. A valid mobile number must have 10 digits. The given code will help you to apply the length validation on user input: 1. $mobileno = strlen ($_POST ["Mobile"]);

```
2. $length = strlen ($mobileno);
3.
4. if ( $length < 10 && $length > 10) {
5. $ErrMsg = "Mobile must have 10 digits.";
6. echo $ErrMsg;
7. } else {
8. echo "Your Mobile number is: " .$mobileno;
9. }
```

Validate URL

The below code validates the URL of website provided by the user via HTML form. If the field does not contain a valid URL, the code will display an error message, i.e., "URL is not valid".

```
1. $websiteURL = $_POST["website"];
2. if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a-z0-
   9+&@#\/%=~_|]/i",$website)) {
3. $websiteErr = "URL is not valid";
4. echo $websiteErr;
5. } else {
6. echo "Website URL is: " .$websiteURL;
7. }
```

Button Click Validate

The below code validates that the user click on submit button and send the form data to the server one of the following method - get or post.

```
1. if (isset ($_POST['submit']) {
2. echo "Submit button is clicked.";
3. if ($_SERVER["REQUEST_METHOD"] == "POST") {
4. echo "Data is sent using POST method ";
5. }
6. } else {
7. echo "Data is not submitted";
8. }
9.
```

Note: Remember that validation and verification both are different from each other. Now we will apply all these validations to an HTML form to validate the fields. Thereby you can learn in detail how these codes will be used to validation form.

Create a registration form using HTML and perform server-side validation. Follow the below instructions as given:

Create and validate a Registration form

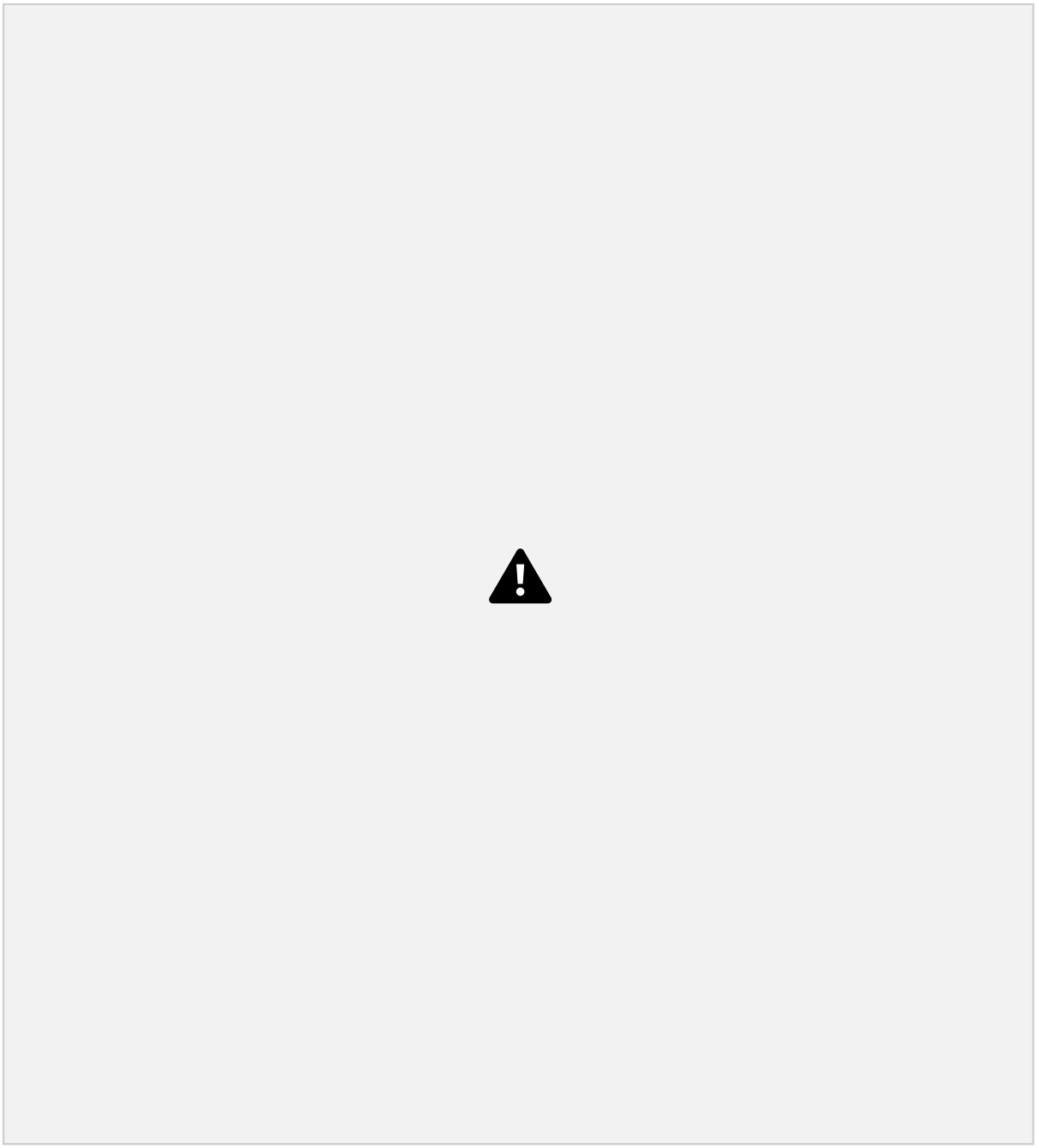
```
1. <!DOCTYPE html>
2. <html>
3. <head>
4. <style>
5. .error {color: #FF0001;}
6. </style>
```

```
7. </head>
8. <body>
9.
10. <?php
11. // define variables to empty values
12. $nameErr = $emailErr = $mobilenoErr = $genderErr = $websiteErr = $agreeErr = "";
13. $name = $email = $mobileno = $gender = $website = $agree = "";  14.
15. //Input fields validation
16. if ($_SERVER["REQUEST_METHOD"] == "POST") {
17.
18. //String Validation
19. if (emptyempty($_POST["name"])) {
20. $nameErr = "Name is required";
21. } else {
22. $name = input_data($_POST["name"]);
23. // check if name only contains letters and whitespace
24. if (!preg_match("/^[a-zA-Z ]*$/",$name)) {
25. $nameErr = "Only alphabets and white space are allowed";  26. }
27. }
28.
29. //Email Validation
30. if (emptyempty($_POST["email"])) {
31. $emailErr = "Email is required";
32. } else {
33. $email = input_data($_POST["email"]);
34. // check that the e-mail address is well-formed
35. if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
36. $emailErr = "Invalid email format";
37. }
38. }
39.
40. //Number Validation
41. if (emptyempty($_POST["mobileno"])) {
42. $mobilenoErr = "Mobile no is required";
43. } else {
44. $mobileno = input_data($_POST["mobileno"]);
45. // check if mobile no is well-formed
46. if (!preg_match ("/^[0-9]*$/", $mobileno) ) {
47. $mobilenoErr = "Only numeric value is allowed.";
48. }
49. //check mobile no length should not be less and greator than 10  50.
if (strlen ($mobileno) != 10) {
51. $mobilenoErr = "Mobile no must contain 10 digits.";
52. }
53. }
54.
55. //URL Validation
56. if (emptyempty($_POST["website"])) {
57. $website = "";
58. } else {
59. $website = input_data($_POST["website"]);
```

```
60. // check if URL address syntax is valid
61. if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a
    z0-9+&@#\/%=~_|]/i",$website)) {
62. $websiteErr = "Invalid URL";
63. }
64. }
65.
66. //Empty Field Validation
67. if (empty empty ($_POST["gender"])) {
68. $genderErr = "Gender is required";
69. } else {
70. $gender = input_data($_POST["gender"]);
71. }
72.
73. //Checkbox Validation
74. if (!isset($_POST['agree'])){
75. $agreeErr = "Accept terms of services before submit.";  76. }
else {
77. $agree = input_data($_POST["agree"]);
78. }
79. }
80. function input_data($data) {
81. $data = trim($data);
82. $data = stripslashes($data);
83. $data = htmlspecialchars($data);
84. return $data;
85. }
86. ?>
87.
88. <h2>Registration Form</h2>
89. <span class = "error">* required field </span>
90. <br><br>
91. <form method="post" action="<?php echo
    htmlspecialchars($_SERVER["PHP_SELF "]); ?>" >
92. Name:
93. <input type="text" name="name">
94. <span class="error">* <?php echo $nameErr; ?> </span>
95. <br><br>
96. E-mail:
97. <input type="text" name="email">
98. <span class="error">* <?php echo $emailErr; ?> </span>
99. <br><br>
100. Mobile No:
101. <input type="text" name="mobileno">
102. <span class="error">* <?php echo $mobilenoErr; ?> </span>  103.
<br><br>
104. Website:
105. <input type="text" name="website">
106. <span class="error"><?php echo $websiteErr; ?> </span>  107.
<br><br>
108. Gender:
```

```
109. <input type="radio" name="gender" value="male"> Male  110. <input
type="radio" name="gender" value="female"> Female  111. <input
type="radio" name="gender" value="other"> Other  112. <span
class="error">* <?php echo $genderErr; ?> </span>  113. <br><br>
114. Agree to Terms of Service:
115. <input type="checkbox" name="agree">
116. <span class="error">* <?php echo $agreeErr; ?> </span>  117.
<br><br>
118. <input type="submit" name="submit" value="Submit">  119.
<br><br>
120. </form>
121.
122. <?php
123. if(isset($_POST['submit'])) {
124. if($nameErr == "" && $emailErr == "" && $mobilenoErr == "" && $genderEr r == ""
    && $websiteErr == "" && $agreeErr == "") {
125. echo "<h3 color = #FF0001> <b>You have sucessfully registered.</b> </ h3>";
126. echo "<h2>Your Input:</h2>";
127. echo "Name: " .$name;
128. echo "<br>";
129. echo "Email: " .$email;
130. echo "<br>";
131. echo "Mobile No: " .$mobileno;
132. echo "<br>";
133. echo "Website: " .$website;
134. echo "<br>";
135. echo "Gender: " .$gender;
136. } else {
137. echo "<h3> <b>You didn't filled up the form correctly.</b> </h3>";  138. }
139. }
140. ?>
141.
142. </body>
143. </html>
```

When the above code runs on a browser, the output will be like the screenshot below:



Fill the registration form and click on the Submit button. If all required information is provided correctly, the output will be displayed on the same page below the submit button. See the screenshot below: